

Logica booleana

Dario Pescini

Università degli Studi di Milano-Bicocca

Dipartimento di Statistica e Metodi Quantitativi

`dario.pescini@unimib.it`

≡ Il risultato di un confronto

Gli operatori di confronto `>` `<` `==` `!=` li avete già visti. Quello che **producono**, no:

- `umidita > 90` non è una domanda: è un'espressione, e ha un valore
- quel valore ha due sole possibilità: `True` e `False`
- maiuscola obbligatoria, e senza virgolette: non sono stringhe
- si mette in una variabile come qualunque altro valore

```
umidita = 93
afoso = umidita > 90
print(afoso)      # True
print(type(afoso)) # il tipo è bool
```

≡ bool: il quarto tipo

Accanto a `int`, `float` e `str`, con la sua funzione di conversione:

- `bool(0)` vale `False`; `bool(3)` e `bool(-1)` valgono `True`
- `bool("")` vale `False`; `bool("ciao")` vale `True`
- la regola: vuoto o zero è falso, tutto il resto è vero

La trappola

`bool("False")` vale `True`: è una stringa, e non è vuota. Quello che c'è scritto dentro non conta.

☰ Che cosa fanno

- ricevono in **ingresso** valori booleani
- restituiscono **un valore** ancora booleano
- sono tre: **not**, **and**, **or**

$$F : \{\text{True, False}\}^n \rightarrow \{\text{True, False}\}$$

Non confonderli con i confronti

Gli operatori di confronto **producono** booleani a partire da numeri o stringhe. Questi tre **combinano** booleani fra loro.

≡ not

Rovescia il valore che riceve: uno solo in ingresso.

$$\text{not} : \{\text{True}, \text{False}\} \rightarrow \{\text{True}, \text{False}\}$$

not

A	not(A)
V	F
F	V

value = not A

and

Vero **solo** quando lo sono tutti e due.

$$\text{and} : \{\text{True}, \text{False}\}^2 \rightarrow \{\text{True}, \text{False}\}$$

and

	V	F
V	V	F
F	F	F

value = A and B

≡ or

Falso **solo** quando sono falsi tutti e due.

$$\text{or} : \{\text{True, False}\}^2 \rightarrow \{\text{True, False}\}$$

or

	V	F
V	V	V
F	V	F

value = A or B

≡ L'or della logica include

- in italiano «o» spesso **esclude**: al bar, «caffè o cornetto» vuol dire uno dei due
- l'or della logica booleana **include**: è vero anche quando sono veri tutti e due
- è la riga in alto a sinistra della tavola, $\vee$ or $\vee$ che dà $\vee$

Lo sconto

«Hanno lo sconto gli studenti o gli under 26.»

Uno studente di 22 anni lo sconto ce l'ha: rientra in tutti e due i casi, e questo non lo esclude.

≡ XOR, l'«o» che esclude

	V	F
V	F	V
F	V	F

- vero quando i due valori **differiscono**
 - falso quando sono uguali, veri o falsi che siano
-
- in Python **non** esiste una parola `xor`
 - sui booleani si scrive con **!=**: `A != B` è vero esattamente quando A e B sono diversi

≡ Precedenza

Quando in un'espressione ce n'è più di uno, l'ordine è questo:

① not

② and

③ or

not a and b	si legge	(not a) and b
a or b and c	si legge	a or (b and c)

Il consiglio

Le parentesi **scrivetele lo stesso**. Non costano niente, e chi legge — compreso voi fra un mese — non deve ricordarsi la regola.

≡ Un esempio sui dati

Le misure meteo di Milano: quali giorni sono **afosi**?

```
afoso = temperatura > 30 and umidita > 70
```

- `temperatura > 30` è un confronto, e produce un booleano
- `umidita > 70` è un altro confronto, e ne produce un altro
- `and` li combina: vero solo dove valgono tutti e due

Con `or` al posto di `and` si selezionano i giorni caldi *oppure* umidi: molti di più, e non è la stessa domanda.

≡ Negare una condizione doppia

Negare un and non dà un and. Le due leggi di **De Morgan**:

$\text{not } (a \text{ and } b)$ equivale a $(\text{not } a) \text{ or } (\text{not } b)$

$\text{not } (a \text{ or } b)$ $(\text{not } a) \text{ and } (\text{not } b)$

- il not **entra** nelle parentesi, su ciascuna delle due condizioni
- e l'operatore in mezzo **si ribalta**: and diventa or, or diventa and
- è la parte controintuitiva: ci si aspetta «e», ed è «o»

≡ A che cosa serve

I giorni **non** afosi:

come viene `not (temperatura > 30 and umidita > 70)`

si riscrive `temperatura <= 30 or umidita <= 70`

- in italiano: «**non** fa caldo **oppure** non è umido»
- basta che **una** delle due condizioni salti perché il giorno non sia afoso
- la seconda riga è la forma che si scrive davvero in un filtro: niente parentesi, niente not

≡ and e or sono pigri

- and: appena trova un False il risultato è deciso, e **si ferma**
- or: appena trova un True, idem
- quindi la seconda condizione **può non venire valutata mai**

```
x != 0 and 1/x > 2
```

- se x vale 0, la prima è falsa e la divisione **non si fa**
- per $1/0$ non c'è risposta: la pigrizia è quello che ci protegge
- scritto al contrario, $1/x > 2$ and $x \neq 0$ va a sbattere

≡ Dalla frase all'espressione

«ha la tessera e spende più di 50»	tessera and spesa > 50
«è di primo o di secondo anno»	anno == 1 or anno == 2
«non ha risposto»	not risposto
«ha risposto ma non è iscritto»	risposto and not iscritto

- «ma», «però», «mentre» sono and: la lingua cambia parola, la logica no
- ogni condizione vuole il suo **soggetto**, anche quando l'italiano lo sottintende

≡ Dall'espressione alla frase

`not (voto >= 18)`

«non ha superato l'esame»

`piove or nevicata`

«c'è precipitazione»

`not piove and not nevicata`

«è asciutto»

- la prima si riscrive senza not: `voto < 18`
- la terza è la seconda negata — De Morgan: `not (piove or nevicata)`
- saper leggere un'espressione ad alta voce è il modo più veloce di accorgersi che è **sbagliata**

≡ Tre errori da aspettarsi

- $x > 0$ and $x < 10$ **non si scrive**: la seconda condizione ha perso il soggetto. Va scritto $x > 0$ and $x < 10$
- `afoso == True` è ridondante: `afoso` è **già** un booleano, e basta scrivere `afoso`
- «gli studenti di primo e secondo anno» si traduce con `or`: nessuno studente è di due anni insieme, e con `and` non si seleziona **nessuno**

Il punto

- un confronto **produce** un booleano; not, and e or li **combinano**
- l'or della logica **include**; quello che esclude è XOR, e in Python si scrive !=
- il not che entra in una parentesi **ribalta** l'operatore — De Morgan
- and e or si fermano appena il risultato è deciso
- nel dubbio, **parentesi**

Esistono anche & e |, che lavorano bit a bit sugli interi: qui non servono, e li ritroverete quando filtrerete una tabella con pandas.